Chapter 6 Exercises

State decimal or binary units, request size, concurrency, and failure assumptions. RAID capacities exclude spares unless specified.

CH06-Q01 Disk geometry and access time

A legacy disk has 10, 000 cylinders, four surfaces, 500 sectors/track, 512 bytes/sector, and 12, 000 RPM. Mean seek is 4.5 ms, controller overhead 0.10 ms, and queueing is ignored.

1. Compute logical sector payload in GB and GiB.
2. Compute rotation period and mean rotational delay.
3. Compute a sector's transfer time.
4. Compute a random sector read's mean latency, then total time for 100 adjacent sectors in one track, combined into one continuous request with no additional rotation or setup.

CH06-Q02 Availability and repair

MTTF is 100, 000 hours and MTTR four hours.

1. Compute long-run availability.
2. Estimate downtime minutes in 8760 hours using that fraction.
3. Reduce MTTR to half an hour and find the new downtime and reduction.
4. Does this establish at most one outage a year?

CH06-Q03 SSD write amplification

A host writes 120 GB/day; NAND writes 384 GB/day. Physical NAND capacity for the simplified endurance budget is 400 GB, with 3000 program/erase cycles.

1. Find WA.
2. Estimate total host-write endurance as physical capacity times cycle budget divided by WA.
3. In another workload, 20% of the same physical NAND is spare and host capacity is 320 GB. A larger working set produces observed WA 4. Recompute endurance. Do not attribute this WA increase to spare area alone.
4. Explain collection costs and design a benchmark with explicit free space, access pattern, and counters.

Chapter 6 Exercises (continued)

CH06-Q04 Polling and interrupts

At 20, 000 operations/s, a status poll costs 100 CPU cycles and an interrupt handler 1500. CPU frequency is 3 GHz. Design P performs exactly one immediately successful poll per operation, with no additional waiting. Design I uses one handler per completion.

1. Compute cycles/s for each overhead.
2. Compute CPU utilization fractions.
3. Does adding 100 runnable threads alone change the ranking under those fixed assumptions? What changes if repeated waiting polls are introduced?
4. Propose a hybrid policy and its required measurements.

CH06-Q05 MMIO and DMA

DMA setup costs 500 cycles and moves 64 MiB in 20 ms. Programmed I/O costs four CPU cycles per four bytes plus ten cycles per transaction. Use 16-byte CPU transactions and a 3-GHz CPU.

1. Compute DMA payload bandwidth.
2. Compute setup time.
3. Compute programmed-I/O CPU cycles and CPU time.
4. State bandwidth, translation, coherence, ordering, and lifetime conditions still needed by DMA.

CH06-Q06 I/O bottlenecks

Six SSDs each support 80, 000 IOPS. The HBA supports 120, 000, memory path 300, 000 equivalent IOPS, and CPU/driver 150, 000.

1. Compute aggregate device capacity.
2. Identify the first predicted ceiling.
3. Does adding two identical SSDs raise it?
4. Propose three interventions and identify the remaining or next bottleneck. Distinguish an intervention that raises the present ceiling from one that improves a different metric.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter 6 Exercises (continued)

CH06-Q07 RAID capacity and failures

Use eight equal 8-TB disks.

1. Give nominal RAID 0, 5, 6, and 10 capacities.
2. Give their disk-failure tolerance models.
3. Count reads and writes for an uncached RAID 5 small read-modify-write.
4. Count accesses for an analogous RAID 6 data/P/Q update.

CH06-Q08 Rebuild traffic

A 12-member RAID 5 uses 10-TB disks and is 90% full. Assume occupancy-aware rebuilding of uniformly distributed nine-TB valid extents per disk. Each lost block requires a block read from all eleven survivors. A shared path supports 1 GB/s aggregate surviving reads plus replacement writes. Use decimal TB, ignore overhead, and compare whole-disk rebuilding that cannot skip unused extents.

1. Find surviving-read bytes for the occupancy-aware case.
2. Add replacement writes and derive the shared-path lower-bound time. Also give the full-disk result.
3. Why can RAID 6 reconstruction not be estimated solely by multiplying the RAID 5 result by 10/11?
4. Why does one MTTF value not determine additional failure exposure during rebuilding?

CH06-Q09 Network transfer

Two servers share a 10-Gbit/s link with 80% payload efficiency. Transfer 64 MiB in four-MiB batches. The fifteen inter-batch gaps each require 0.20 ms; initial setup is excluded unless stated separately.

1. Find raw payload bits.
2. Find effective payload bandwidth.
3. Find transfer plus inter-batch time and the result with one additional initial setup.
4. Can 100 concurrent streams divide this time by 100? Explain bandwidth and queueing limits.

Chapter 6 Exercises (continued)

CH06-Q10 Queueing and design targets

Assume an FCFS M/M/1 server with Poisson arrivals, exponential service, unlimited waiting room, and steady state. Mean service is 4 ms, lambda=200/s, and each request 4 KiB.

1. Find mu, rho, W, Wq, L, and payload bandwidth.
2. For mean response≤10 ms, find maximum lambda, utilization, and mean requests in system.
3. Give a numeric counterexample where only arrival rate rises and latency increases. Does Little's law require Poisson arrivals?

CH06-Q11 Parity updates and write hole

RAID5 has four data blocks and one parity block, each 4 KiB, no caches/protocol overhead. Representative values are D0=1010, D1=1100, D2=0110, D3=1111.

1. Compute P and P' after D1 becomes 0101, both by full recomputation and shortcut.
2. Compare RMW and reconstruct-write block operations/payload for k=1 and k=3 updated blocks, new data supplied by host.
3. If new D1 is durable but parity is old at power loss, which invariant fails? Why does duplicate parity not guarantee update atomicity?

CH06-Q12 Two P+Q erasures

Use GF(16), modulus 10011, coefficients 1/2/4, and D0=5, D1=7, D2=9.

1. Compute P/Q, showing reduction for multiplication by 2 and 4.
2. Lose D_0, D_2. Derive residual equations and recover them. You may use $5^{-1} = 11$ in the stated finite field, but verify $5 \cdot 11 = 1$.
3. Why are known-position erasures different from arbitrary silent corruption?

Chapter 6 Exercises (continued)

CH06-Q13 Rebuild and conditional failure

An eight-disk RAID5 lost one disk. Rebuild 6 TB, reading one block from each of seven survivors per replacement block. Reads/writes share aggregate 2 GB/s. Survivors have independent fixed hazards 10^-5/h. Actual rebuild time equals the traffic lower bound; ignore media errors.

1. Find read/write/total traffic and hours.
2. Compute the conditional probability of another disk failure during this interval.
3. If 2 GB/s instead denotes replacement output, find time and required survivor-read bandwidth. Why is the probability not annual array data-loss risk?

CH06-Q14 NAND reclamation and writable pages

A victim erase block has eight pages: five live and three invalid. A destination block has eight writable free pages. Copy all live pages to the destination before erasing the victim. Other blocks do not change and all pages have the same size.

1. Count the two blocks' writable pages before copying, after copying, and after erasing. Find the net gain.
2. If the host then writes three new pages, count page programs due to copying and host writes, and calculate this batch's write amplification. Does an erase count as a written page?
3. If only four free pages are available for copying, can the victim immediately be erased under this sequence? Explain.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Chapter 6 Exercises (continued)

CH06-Q15 Split transactions and ordering

A shared bus carries at most one request or response in each slot. Assume the device can process multiple requests: A's request is in [0,1), B's request in [1,2), B's response in [4,5), and A's response in [6,7).

1. List the occupied slots, calculate bus utilization over [0,7), and give each request-start-to-response-end latency.
2. Why identify outstanding transactions? Does B completing first imply that all interfaces permit response reordering?
3. Why is this schedule infeasible if the device accepts only one outstanding transaction?

References and attribution

David A. Patterson and John L. Hennessy, Computer Organization and Design: The Hardware/Software Interface, Revised Fourth Edition, Chapter 6. Lin Cing, handwritten computer organization notes; computer organization and structure (CSO) course slides.

Original educational synthesis by Lin Cing, licensed under CC BY 4.0.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study